LSC LAUREL SOFTWARE COMPANY

Web development Task

Task assign to: Hammad Mehmood and Muhammad Muneeb

Task Assignment: Interactive Bento Grid Dashboard

Overview

Your task is to transition your static HTML/CSS skills into a modern frontend framework by building a highly interactive, responsive **Personal Productivity Dashboard**.

Instead of a traditional linear layout, you will design this dashboard using a **Bento Grid** a modern UI design trend featuring a collection of tiles of varying sizes structurally aligned in a tight grid.

Technical Stack Requirements

- **Framework:** React (with TypeScript) / Vue / Svelte *(Specify your team's preferred framework)*
- **Styling:** Tailwind CSS or custom CSS (must utilize **CSS Grid** for the layout)
- **Icons:** Lucide-react, React Icons, or FontAwesome
- **State Management:** Core framework state primitives (e.g., React useState, Vue ref/reactive)

Feature Requirements & Components

You must break your dashboard down into self-contained, reusable components. Below is the blueprint of what your Bento Grid must contain:

1. The Grid Layout (<DashboardGrid/>)

- Must use **CSS Grid** to create a responsive bento-style layout.
- Minimum of 5 grid items (tiles) of varying column/row spans.
- **Responsiveness:** On mobile screens, the grid must gracefully collapse into a single-column layout. On larger screens, it must show the bento pattern.

2. Digital Clock & Date Tile (<ClockTile/>)

- Displays the current time (HH:MM:SS) and date.
- **Dynamic Update:** The time must update every single second using a JavaScript interval (setInterval).
- *Crucial Framework Concept:* Remember to clear your interval when the component unmounts to prevent memory leaks!

3. Quick Links Tile (<QuickLinksTile/> & <LinkCard/>)

- A section containing a minimum of 4 favorite shortcuts (e.g., GitHub, Google, Slack, StackOverflow).
- **Component Reusability:** The individual links must be rendered using a reusable child component called <LinkCard/>.
- **Props:** You must pass the destination URL, display name, and icon down to each <LinkCard/> via props.

4. Interactive Weather Tile (<WeatherTile/>)

- Displays a city name, temperature, current weather condition (e.g., Sunny, Rainy), and a matching icon.
- **Interactive State:** Include a toggle button inside this tile that switches the temperature display between **Celsius** and **Fahrenheit** dynamically.
- *(Note: You can use hardcoded static data for the weather initially, but the unit conversion state must work).*

5. Theme Switcher Tile (<ThemeToggleTile/>)

- A tile featuring a toggle button to switch the entire dashboard between **Dark Mode** and **Light Mode**.
- Clicking this button should dynamically apply/remove a class (like .dark or data-theme="dark") on the main wrapper or the HTML body, causing all tiles to change their background, text, and border colors instantly.

Key Framework Concepts You Must Demonstrate

- **Component Architecture:** Every tile should be its own file/component. Do not write the entire application in one giant file.
- **Props Usage:** Demonstrating how to make components reusable by passing dynamic data (specifically in the <LinkCard/>).
- **State & Reactivity:** Handling dynamic updates (the clock), toggles (Celsius/Fahrenheit), and global styling triggers (Dark/Light mode) using reactive variables.
- **Side Effects (if using React):** Proper handling of useEffect for setting up and tearing down the clock interval.

Deliverables

Git Repository: A clean, committed GitHub repository containing your code. Code cleanliness and structured folder architecture (e.g., src/components/, src/styles/) will be graded.